

Notebook vs. Percent-Cell Python for Coding Agents

Overview

Interactive data analysis is often taught and reviewed in notebooks, but coding agents have to read, edit, validate, and re-send the working artifact as context during a task. This report tests whether an `.ipynb` notebook is more token-expensive for agents than an equivalent VS Code/Jupyter percent-cell `.py` file, which preserves cell-based interaction while storing the analysis as plain Python text.

We ran two paired agent experiments in which agents analyzed Seattle Public Library checkout data to compare physical and digital borrowing trends over time and produce a short reproducible analysis report. In the controlled direct-edit comparison, one subagent edited and executed an existing notebook while a second subagent edited and executed an equivalent percent-cell Python file. In the agent-native comparison, one subagent used a less constrained notebook workflow that included generating, executing, inspecting, patching, and rerunning notebook artifacts, while the paired subagent used a percent-cell Python workflow. Across both comparisons, the notebook workflow accumulated more total tokens, required more model calls, and took longer. Internal worker names are retained as log labels for auditability, but the report treats workflow as the main unit of comparison.

Main takeaway: both comparisons point in the same direction. Notebook workflows required more model calls, took longer, and accumulated more total task tokens. The exact cost multiplier is a best estimate because prompt caching changes the fresh/cached input split.

1.59x Agent-native total-token ratio
1.94x Controlled total-token ratio
1.25x Agent-native model-call ratio
1.71x Controlled model-call ratio

In both the looser agent-native workflow and the stricter direct-edit workflow, the notebook path required more model turns and accumulated more total session context. This workflow-friction result is stronger than any precise universal cost multiplier.

Total tokens are still useful because they measure cumulative context traffic. Fresh input and observed cost are useful too, but they are cache-sensitive and should be presented as estimates.

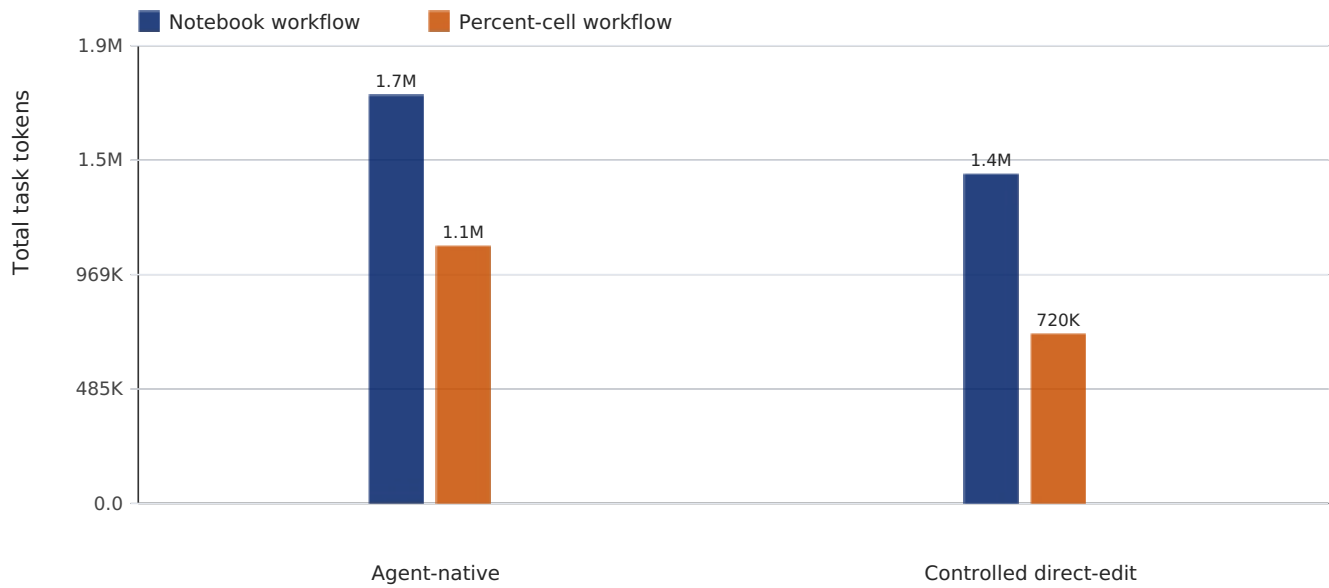


Figure: Total task tokens for notebook and percent-cell workflows within each experiment. **Takeaway:** the notebook workflow used more total tokens in both the agent-native and controlled comparisons.

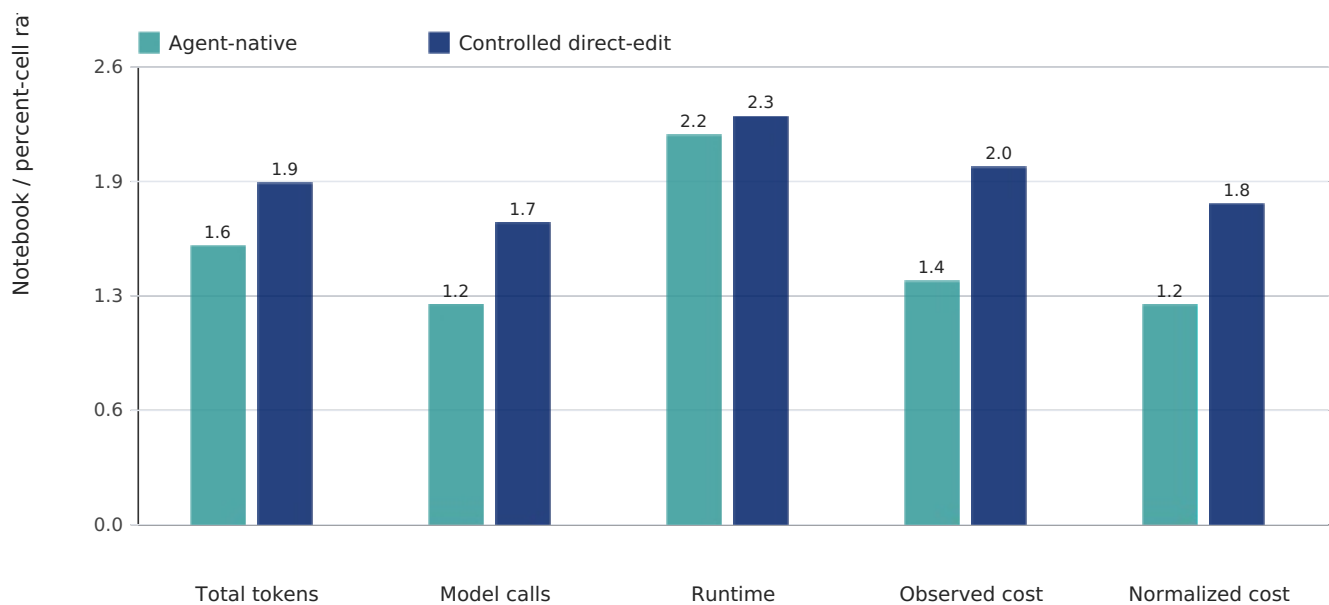


Figure: Notebook-to-percent-cell ratios for total tokens, model calls, runtime, and cost estimates. **Takeaway:** the notebook workflow is consistently above parity, while the exact cost ratio is more cache-sensitive than the workflow and token-count ratios.

Controlled Direct-Edit Comparison

This section isolates the file-format question as closely as this experiment allowed. Two subagents were given the same data, the same analysis question, and similar deliverable requirements; the main intended difference was whether the durable working source was an `.ipynb` notebook or a percent-cell `.py` file.

The notebook worker edited an existing notebook, executed it, and inspected saved notebook outputs. The percent-cell worker edited an equivalent Python file and inspected terminal or saved outputs. This comparison is the strongest evidence in the report because it controls the task framing more tightly than the agent-native run.

Workflow	Log label	Total tokens	Fresh input	Model calls	Runtime	Observed cost	Source artifact tokens
Direct .ipynb	Carson	1,395,748	129,036	24	8m 30s	\$1.61	84,753
Percent-cell .py	Confucius	719,688	51,000	14	3m 40s	\$0.79	1,653

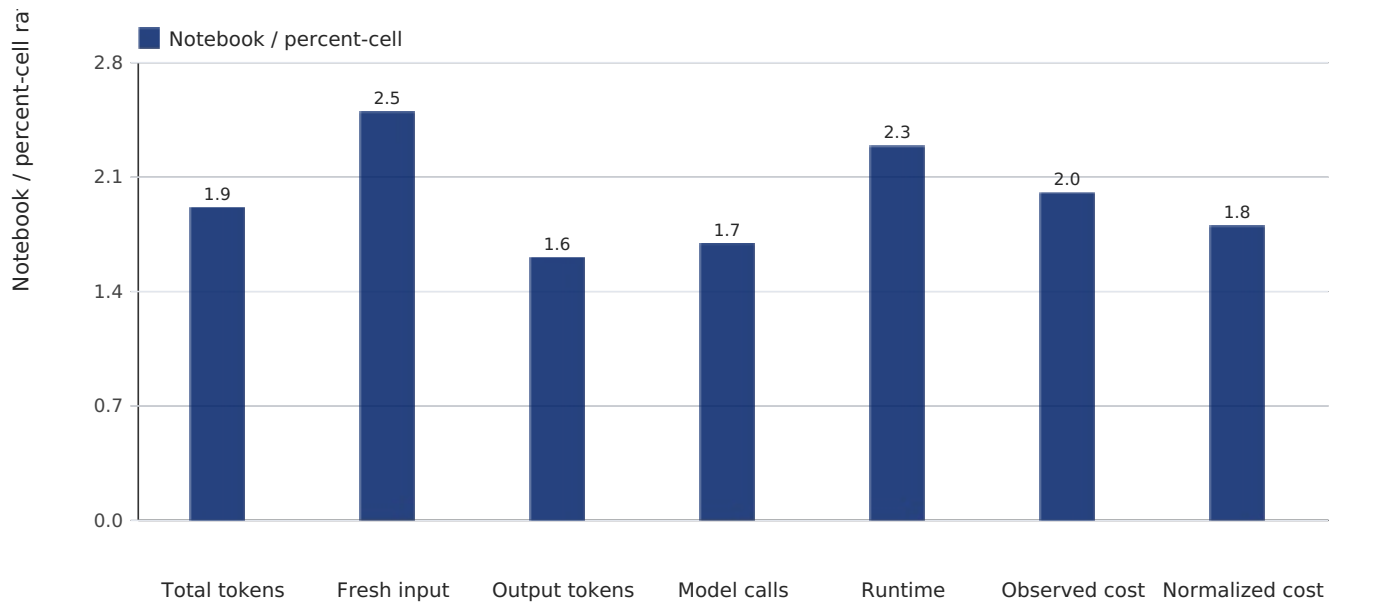


Figure: Controlled direct-edit notebook-to-percent-cell ratios across task metrics. **Takeaway:** the notebook worker required more calls, more elapsed time, more total tokens, and more fresh input than the percent-cell worker.

Tool-Call Categories

Categories group similar work, so source edits or output inspections do not need to use identical commands to be compared.

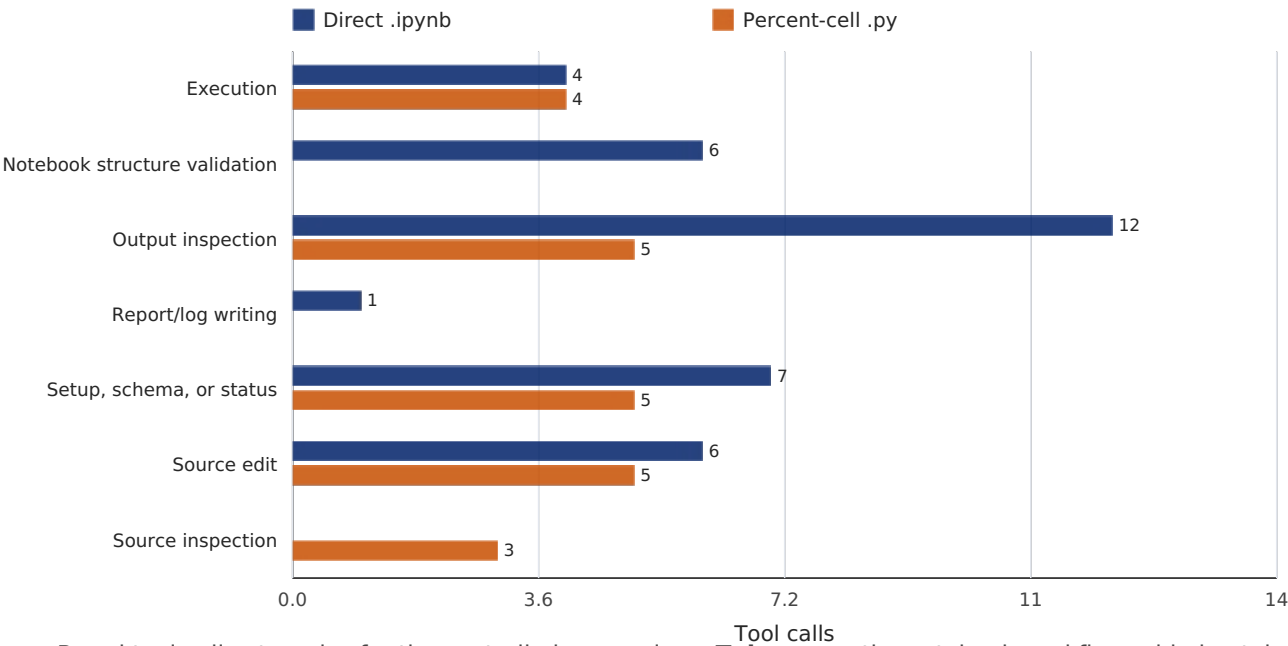


Figure: Broad tool-call categories for the controlled comparison. **Takeaway:** the notebook workflow added notebook-specific structure validation and more output-inspection work, which helps explain the higher call count.

Per-Call Token Delta

Direct .ipynb workflow

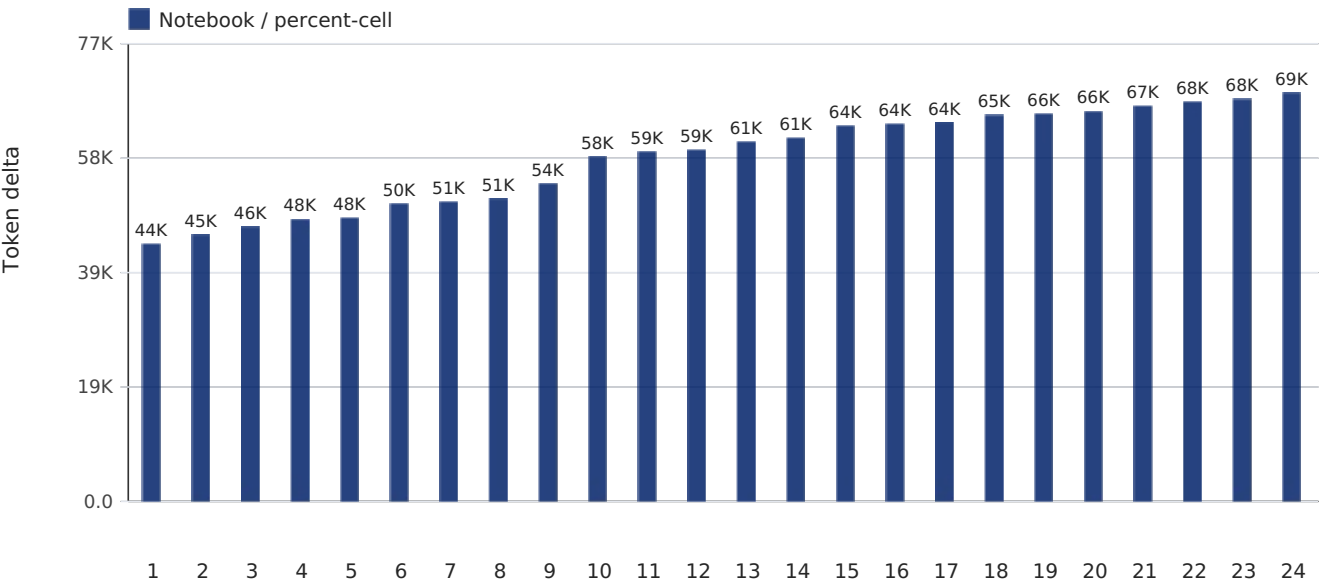


Figure: Per-call token delta for the controlled notebook worker. **Takeaway:** the notebook run kept accumulating large per-call context as the task progressed.

Percent-cell .py workflow

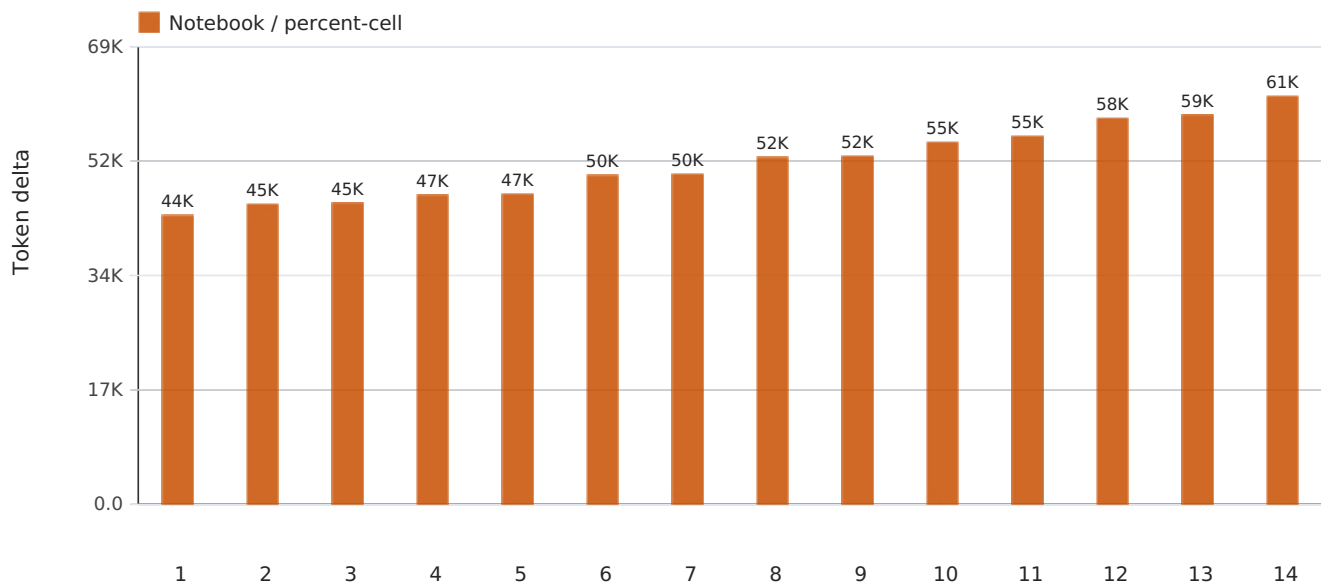


Figure: Per-call token delta for the controlled percent-cell worker. **Takeaway:** the percent-cell run had fewer calls overall, shortening the cumulative token path through the task.

Input Composition

Direct .ipynb workflow

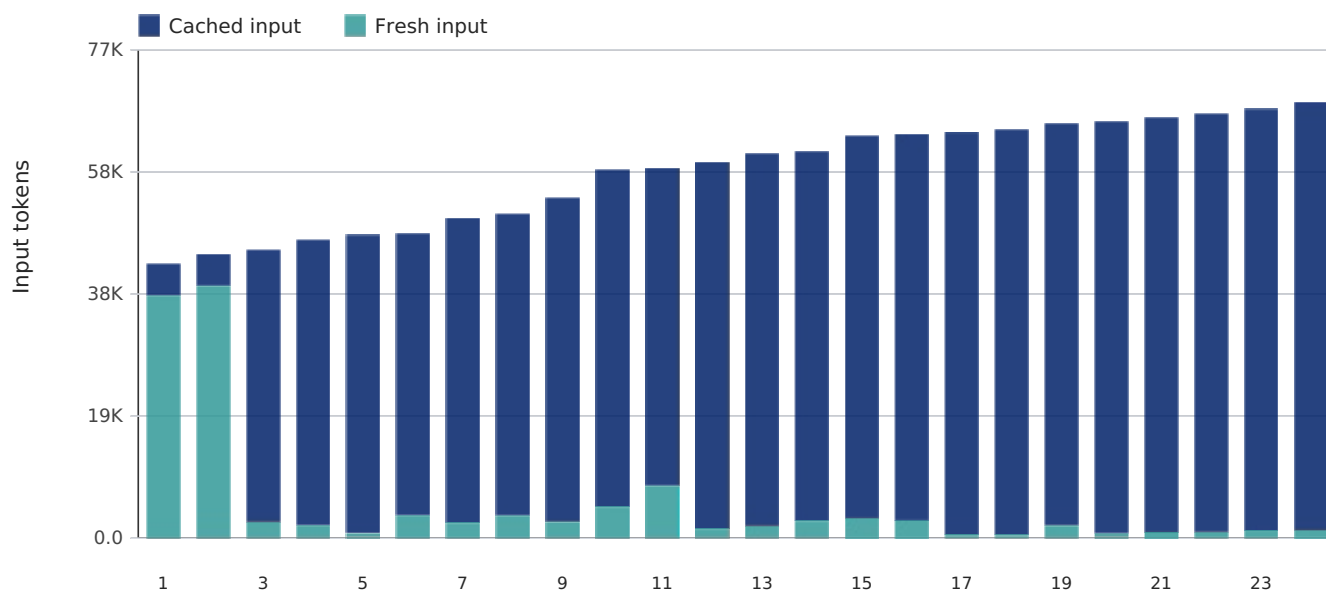


Figure: Cached and fresh input tokens by model call for the controlled notebook worker. **Takeaway:** most later input was cached, but the notebook run still paid for repeated long context and several fresh-input spikes.

Percent-cell .py workflow

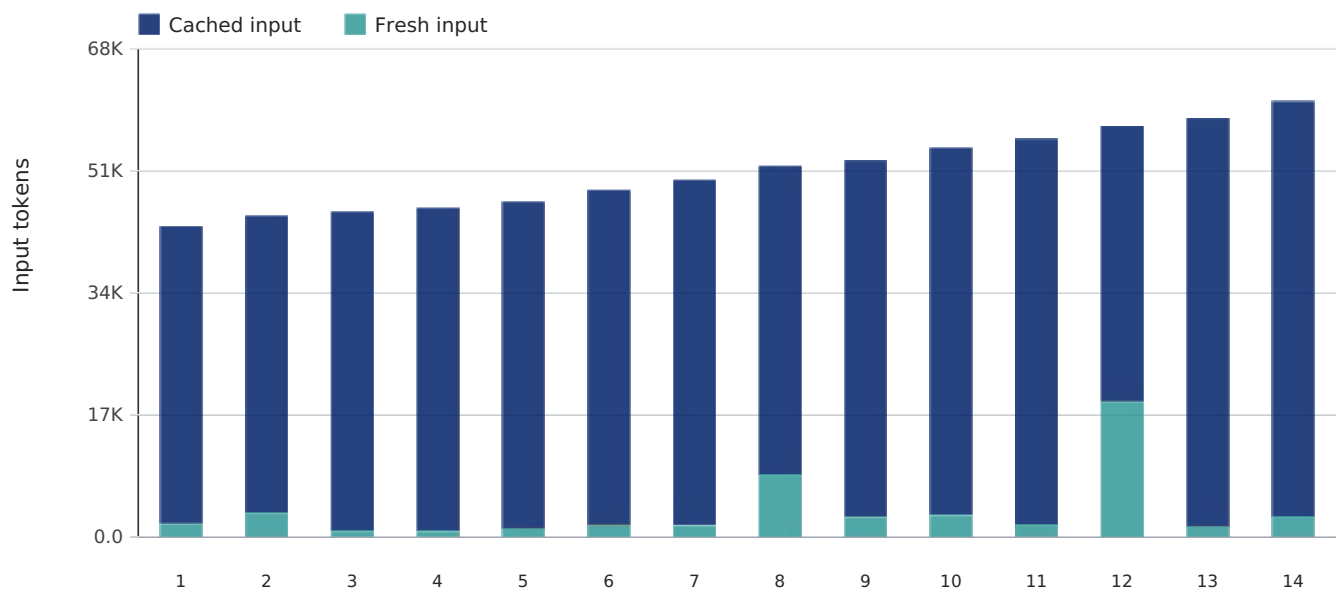


Figure: Cached and fresh input tokens by model call for the controlled percent-cell worker. **Takeaway:** the percent-cell workflow had fewer model calls and less cumulative fresh input.

Tool-Call Category Table

Workflow	Category	Tool calls
direct ipynb	direct notebook edit	6
direct ipynb	environment check	1
direct ipynb	filesystem/navigation	4
direct ipynb	final status check	1
direct ipynb	notebook execution	4
direct ipynb	notebook output extraction	11
direct ipynb	notebook report/log write	1
direct ipynb	notebook structure validation	6
direct ipynb	output artifact inspection	1
direct ipynb	schema reference	1
percent-cell py	environment check	1
percent-cell py	filesystem/navigation	2
percent-cell py	final status check	1
percent-cell py	output artifact inspection	5
percent-cell py	percent-cell execution	4
percent-cell py	percent-cell source edit	5
percent-cell py	percent-cell source inspection	3
percent-cell py	schema reference	1

Agent-Native Comparison

This section uses the earlier, less constrained paired run to ask what happens when the notebook worker is allowed to follow a more natural agent workflow. That notebook workflow included generating notebook structure, executing with notebook tooling, inspecting saved outputs, patching, and rerunning notebook artifacts.

Because the workers had more freedom, this run is less controlled than the direct-edit comparison. Its value is corroboration: it tests whether the same direction appears when the notebook worker behaves more like an unconstrained coding agent might behave in practice.

Workflow	Log label	Total tokens	Fresh input	Model calls	Runtime	Observed cost	Source artifact tokens
Agent-native executed .ipynb	Boyle	1,730,973	122,627	25	10m 29s	\$1.68	81,758
Agent-native percent-cell .py	Laplace	1,091,505	83,054	20	4m 44s	\$1.22	1,734

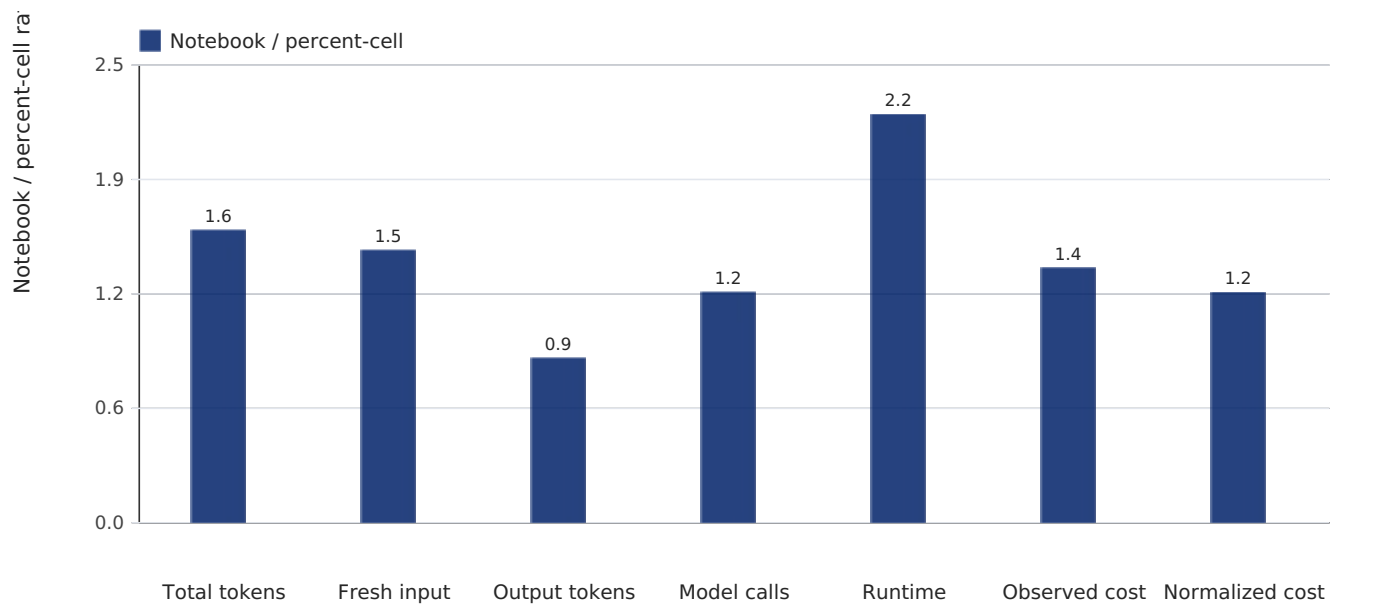


Figure: Agent-native notebook-to-percent-cell ratios across task metrics. **Takeaway:** even in the less constrained run, the notebook workflow used more total tokens, more model calls, more time, and higher estimated cost.

Tool-Call Categories

The agent-native logs mostly expose shell and patch tools, so this chart uses broad command-intent categories.

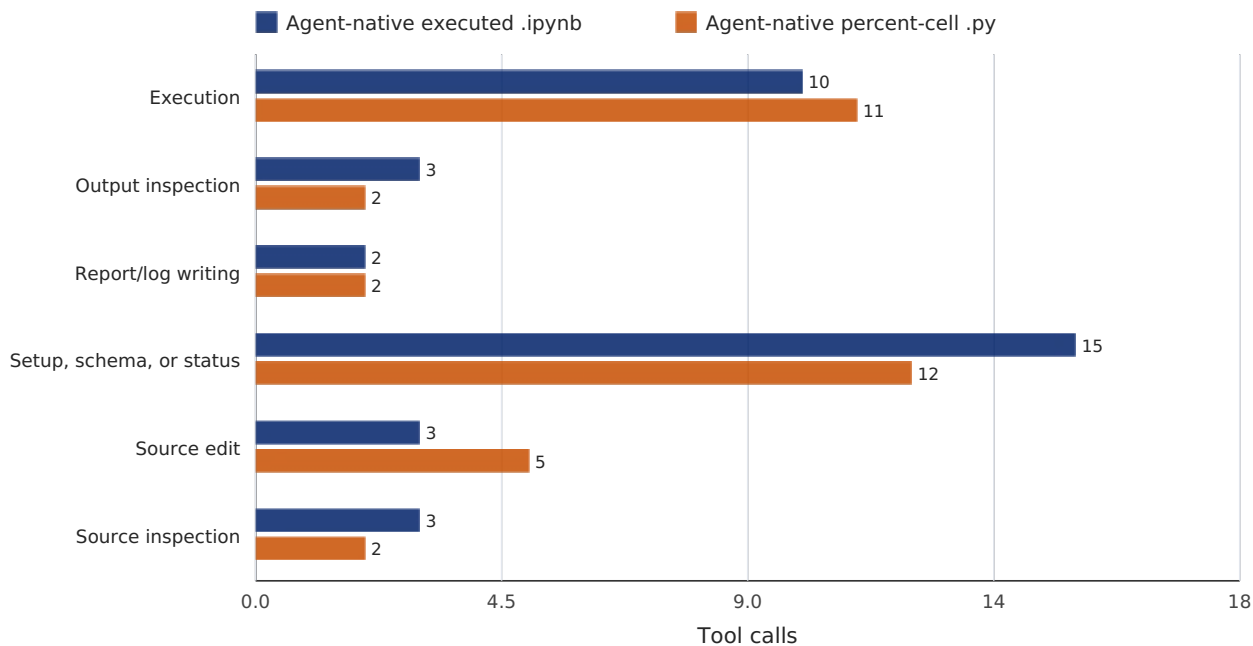


Figure: Broad command-intent categories for the agent-native comparison. **Takeaway:** the agent-native notebook run spent more calls on setup/status and output or execution handling, while broad edit and report-writing work was similar in scale.

Raw Tool Calls

Workflow	Log label	Tool	Calls
Agent-native executed .ipynb	Boyle	apply_patch	5
Agent-native executed .ipynb	Boyle	exec_command	30
Agent-native executed .ipynb	Boyle	write_stdin	1
Agent-native percent-cell .py	Laplace	apply_patch	7
Agent-native percent-cell .py	Laplace	exec_command	27

Cache And Cost Notes

This section separates total session traffic from cache-sensitive cost estimates. The raw token totals show how much input and output accumulated across model calls. API-equivalent cost depends on how much input was classified as fresh versus cached, which varied across first calls in a way that should not be over-interpreted as a file-format effect.

The normalization table treats the notebook first calls as if they had the same common first-call cache pattern seen in the percent-cell runs. This sensitivity check helps show whether the conclusion depends entirely on the first-call cache difference.

Prompt caching changes the split between fresh and cached input. It does not change total input tokens, but it does affect estimated API cost. In these runs, the percent-cell first calls commonly had about 43K input tokens with about 41K cached, while the notebook first calls had similar input length but only about 5K cached.

The table below normalizes each notebook first call to the common 41,344 cached-token first-call pattern as a sensitivity check rather than a claim about what billing should have been.

Comparison	Pair	Observed cost ratio	Normalized fresh-input ratio	Normalized cost ratio
Agent-native	Agent-native executed .ipynb / Agent-native percent-cell .py	1.38x	1.04x	1.25x
Controlled direct-edit	Direct .ipynb / Percent-cell .py	2.03x	1.82x	1.82x

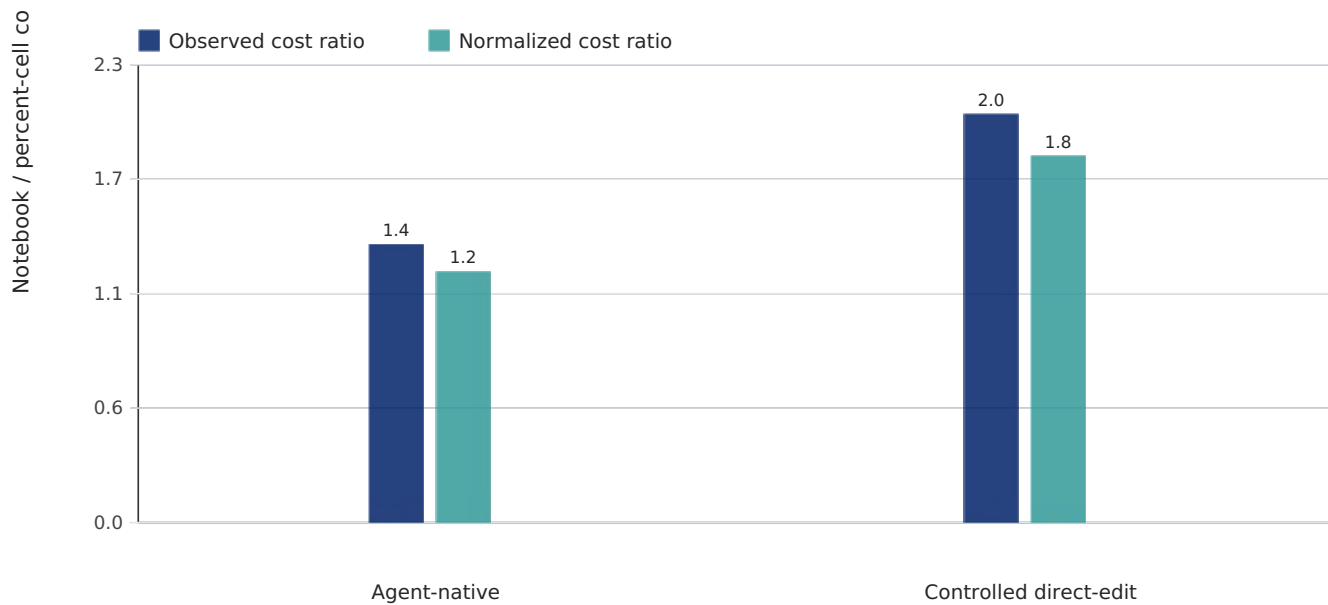


Figure: Observed cost ratios compared with ratios after normalizing notebook first-call cache behavior. **Takeaway:** notebook workflows remain more expensive under this sensitivity check, but the multiplier shrinks when the first-call cache imbalance is reduced.

Interpretation: cost likely moves in the same direction as total tokens and model calls, but the exact cost ratio is less stable than the workflow-turn evidence.

Appendix

This section documents how the report can be audited and rebuilt. The HTML is generated from package-local CSV files and raw Codex JSONL logs; the exact worker prompts are included so readers can inspect the starting conditions for each subagent.

Reproducibility

The page is generated by `scripts/build_report.py` from CSV files and raw JSONL logs included in this package. Plotly is vendored at `assets/plotly-2.35.2.min.js`.

Token Interpretation

`total_tokens` is cumulative accounting across model calls, not unique text. For each call, `input_tokens = cached_input_tokens + non_cached_input_tokens`, and `total_tokens = input_tokens + output_tokens`.

Exact Starting Prompts

The names below are internal worker/log labels preserved so the raw JSONL logs, CSV rows, and prompts can be cross-checked.

Boyle: agent-native notebook prompt

```
You are Worker A in a controlled format-efficiency experiment. You are not alone in the codebase; do not revert  
  
Workspace: /home/jessica-nash/analytics-accelerator/week-2-preparation  
Assigned write scope only: agent-format-experiment/notebook-agent/  
  
Task: Create a small but realistic analysis session as a Jupyter notebook. The goal is to simulate working with  
  
Use this dataset reference first: seattle-public-library/README.md. For the actual analysis, use only seattle-pu  
  
Analysis question: How have Seattle Public Library physical vs digital checkout totals changed across complete y  
  
Deliverables in your assigned directory:  
1. analysis-notebook.ipynb with 5-8 clear cells, including markdown explanation, intermediate displayed outputs,  
2. reports/analysis-report.md: concise markdown report with procedure, checks, findings, conclusion, and referen  
3. reports/assets/: save at least one chart as SVG or PNG and one CSV summary table.  
4. run-log.md: brief note describing how you executed cells and any issues.  
  
Style constraints:  
- Code should be explicit, readable pandas for a student with one semester of Python.  
- Identify row meaning before plotting/interpreting.  
- Use complete years only, requiring 12 months for both physical and digital.  
- Do not do unrelated exploration.  
- Use .venv/bin/python or tools inside .venv only.  
  
Final response: list changed files, execution command(s), and a short summary of findings.
```

Laplace: agent-native percent-cell prompt

```
You are Worker B in a controlled format-efficiency experiment. You are not alone in the codebase; do not revert  
  
Workspace: /home/jessica-nash/analytics-accelerator/week-2-preparation  
Assigned write scope only: agent-format-experiment/py-cell-agent/  
  
Task: Create a small but realistic analysis session as a VS Code percent-cell Python file. The goal is to simula
```

Use this dataset reference first: seattle-public-library/README.md. For the actual analysis, use only seattle-pu

Analysis question: How have Seattle Public Library physical vs digital checkout totals changed across complete y

Deliverables in your assigned directory:

1. analysis-cells.py using VS Code/Jupyter percent-cell format with 5-8 clear cells, including markdown cells an
2. reports/analysis-report.md: concise markdown report with procedure, checks, findings, conclusion, and referen
3. reports/assets/: save at least one chart as SVG or PNG and one CSV summary table.
4. run-log.md: brief note describing how you executed the cells/script and any issues. Since .py cell outputs ar

Style constraints:

- Code should be explicit, readable pandas for a student with one semester of Python.
- Identify row meaning before plotting/interpreting.
- Use complete years only, requiring 12 months for both physical and digital.
- Do not do unrelated exploration.
- Use .venv/bin/python only.

Final response: list changed files, execution command(s), and a short summary of findings.

Carson: direct notebook prompt

You are Worker C in a controlled apples-to-apples format experiment. You are not alone in the codebase; do not r

Workspace: /home/jessica-nash/analytics-accelerator/week-2-preparation

Assigned write scope only: agent-format-experiment/direct-notebook-agent/

Starter artifact: agent-format-experiment/direct-notebook-agent/analysis-notebook.ipynb

Task: Simulate a human-in-the-loop notebook analysis session using direct notebook editing. You must work direct

Use this dataset reference first: seattle-public-library/README.md. For analysis, use only seattle-public-librar

Analysis question: How have Seattle Public Library physical vs digital checkout totals changed across complete y

Required workflow:

1. Inspect the starter notebook structure directly with jq, nbformat, or equivalent targeted commands.
2. Edit the .ipynb directly with nbformat or structured JSON operations. Make incremental notebook edits: add or
3. Execute the notebook with the project environment after meaningful edits, using .venv/bin/jupyter or .venv/bi
4. After execution, inspect targeted cell outputs from the .ipynb JSON, not full notebook dumps. Use the outputs
5. Add or update a final markdown interpretation cell in the notebook based on those inspected outputs.
6. Save at least one chart and one CSV table under agent-format-experiment/direct-notebook-agent/reports/assets/
7. Create agent-format-experiment/direct-notebook-agent/run-log.md with commands run, output cells inspected, an

Constraints:

- No builder script for creating the notebook.
- Do not use terminal Python to compute analysis results outside the accepted notebook. Terminal Python may edit
- Code should be explicit, readable pandas for a student with one semester of Python.
- Identify row meaning before plotting/interpreting.
- Use complete years only, requiring 12 months for both physical and digital.
- Keep the notebook concise: 5-9 cells.

Final response: list changed files, execution commands, output cells inspected, and a short summary of findings.

Confucius: direct percent-cell prompt

You are Worker D in a controlled apples-to-apples format experiment. You are not alone in the codebase; do not r

Workspace: /home/jessica-nash/analytics-accelerator/week-2-preparation

Assigned write scope only: agent-format-experiment/direct-py-cell-agent/

Starter artifact: agent-format-experiment/direct-py-cell-agent/analysis-cells.py

Task: Simulate the same human-in-the-loop analysis session using a VS Code/Jupyter percent-cell Python file. You

Use this dataset reference first: seattle-public-library/README.md. For analysis, use only seattle-public-librar

Analysis question: How have Seattle Public Library physical vs digital checkout totals changed across complete y

Required workflow:

1. Inspect the starter percent-cell file structure directly.
2. Edit analysis-cells.py directly. Make incremental edits: add or modify a few cells at a time.
3. Execute the file with .venv/bin/python after meaningful edits.
4. Inspect targeted terminal output and generated CSV/chart outputs. Use those outputs to decide the interpretat
5. Add or update a final markdown/comment interpretation cell in analysis-cells.py based on inspected outputs.
6. Save at least one chart and one CSV table under agent-format-experiment/direct-py-cell-agent/reports/assets/
7. Create agent-format-experiment/direct-py-cell-agent/run-log.md with commands run, outputs inspected, and brie

Constraints:

- Do not create a separate generator/builder script.
- Do not use terminal Python to compute analysis results outside the accepted .py analysis file. Terminal comman
- Code should be explicit, readable pandas for a student with one semester of Python.
- Identify row meaning before plotting/interpreting.
- Use complete years only, requiring 12 months for both physical and digital.
- Keep the percent-cell file concise: 5-9 cells.

Final response: list changed files, execution commands, outputs inspected, and a short summary of findings.